

1 chapter5: 贪心算法

贪心算法很难有具体可行的方法对其适用性进行判定，但是一般具有 2 个重要的性质：**贪心选择性质**和**最优子结构性质**。

所谓贪心选择性质是指所求问题的整体最优解可以通过一系列局部最优的选择，即贪心选择来达到。这意味着每一步的最优选择不会影响未来的选择，即当前的选择只依赖于当前的状态，而不依赖于未来的状态。

对于一个约束优化问题：

1. **可行解**：如果决策序列是关于一个问题的完整解的形式，且不破坏问题的约束条件，则称它为该问题的可行解。
2. **部分可行解**：如果一个决策序列是问题的完整解形式的一部分，且满足约束条件，则称它为部分可行解。
3. **不可行的（部分）解**：不满足问题约束条件的完整或部分解形式的决策序列，称为不可行的（部分）解。
4. **最优解**：使目标函数值取最优值的可行解，称为最优解。
5. **贪心解**：通过贪心算法得到的可行解，称为贪心解。
6. **贪心解性质**：贪心解不一定是最优解。

1.1 贪心算法思想

Input: 集合 A , 元素数量 n

Output: 解集 $solution$

```
1 贪心算法一般框架
2 Algorithm GREEDY( $A, n$ )
3  $solution \leftarrow \emptyset$ 
4 for  $i \leftarrow 1$  to  $n$  do do
5      $x \leftarrow \text{SELECT}(A)$ 
6     if  $\text{FEASIBLE}(solution, x)$  then
7          $solution \leftarrow \text{UNION}(solution, x)$ 
8     end
9     end if
10 end
11 return ( $solution$ )
```

Algorithm 1: 贪心算法通用框架

1. **贪心选择性质**：通过 SELECT 函数保证每一步选择都是当前最优，且该选择最终能导向全局最优解（需进行数学证明）
2. **最优子结构**：FEASIBLE 函数确保解集的逐步扩展不破坏问题结构。

Input: 带权图 $G = (V, E)$, 权重函数 $w : E \rightarrow \mathbb{R}^+$, 源顶点 $s \in V$

Output: 源点 s 到所有顶点的最短距离 $dist[\cdot]$, 前驱节点数组 $prev[\cdot]$

```
1 foreach 顶点  $v \in V$  do
2    $dist[v] \leftarrow \infty$ ;
3    $prev[v] \leftarrow \text{NIL}$ ;
4 end
5  $dist[s] \leftarrow 0$ ;
6  $Q \leftarrow \text{MINPRIORITYQUEUE}(V)$ ; // 基于  $dist$  值的最小优先队列
7 while  $Q \neq \emptyset$  do
8    $u \leftarrow Q.\text{EXTRACTMIN}()$ ;
9   foreach 邻居顶点  $v \in \text{ADJ}[u]$  do
10    // 松弛操作 (Relaxation)
11     $alt \leftarrow dist[u] + w(u, v)$ ;
12    if  $alt < dist[v]$  then
13       $dist[v] \leftarrow alt$ ;
14       $prev[v] \leftarrow u$ ;
15       $Q.\text{DECREASEKEY}(v, alt)$ ; // 更新队列中  $v$  的优先级
16    end
17  end
18 end
19 return  $(dist, prev)$ ;
```

Algorithm 2: Dijkstra 算法

3. 终止条件: 循环 n 次确保覆盖所有候选元素, 最终解集 solution 必然完备。

贪心算法产生全局最优解的最重要因素是选择策略, 采用的贪心准则/贪心度量。

1.2 贪心算法举例

1.2.1 Dijkstra 算法

- $\lambda[v]$: 算法中顶点 v 的当前距离标记 (对应伪代码中的 $dist[v]$)
- $\delta[v]$: 从源点 1 到 v 的真实最短路径长度
- Y : 未处理的顶点集合 (对应优先队列 Q)
- X : 已处理的顶点集合 ($X = V \setminus Y$)

在 Dijkstra 算法中, 当顶点 y 在第 8 步 (ExtractMin 操作) 被选中且 $\lambda[y]$ 有限时, 有:

$$\lambda[y] = \delta[y]$$

第一个离开 Y 的顶点是源点 1, 初始化时:

$$\lambda[1] = \delta[1] = 0 \quad (\text{显然成立})$$

假设对于所有在 y 之前离开 Y 的顶点 x , 都有:

$$\lambda[x] = \delta[x]$$

下证对 y 也成立。

1. 路径存在性: 由于 $\lambda[y]$ 有限, 必存在从 1 到 y 的路径 P , 其长度为 $\lambda[y]$ 。
2. 关键路径构造: 设 $\pi = 1 \rightarrow \dots \rightarrow x \rightarrow w \rightarrow \dots \rightarrow y$ 是从 1 到 y 的真实最短路径 (π 具有子问题最优性, 否则一定存在一个更短的子路径优化 π), 其中:

- x 是路径 π 上最后一个在 y 之前离开 Y 的顶点
- w 是 x 的后继节点

3. 不等式链:

$$\begin{aligned}\lambda[y] &\leq \lambda[w] \quad (\text{因为 } y \text{ 先于 } w \text{ 被选中}) \\ &\leq \lambda[x] + \text{length}(x, w) \quad (\text{松弛操作保证}) \\ &= \delta[x] + \text{length}(x, w) \quad (\text{归纳假设}) \\ &= \delta[w] \quad (\pi \text{ 的子路径最优性}) \\ &\leq \delta[y] \quad (\text{非负权重下子路径} \leq \text{全路径})\end{aligned}$$

4. 等号成立: 由于 $\lambda[y]$ 是某条路径的实际长度, 而 $\delta[y]$ 是最短长度, 故:

$$\lambda[y] \geq \delta[y]$$

结合上述不等式链, 最终得:

$$\lambda[y] = \delta[y]$$

- 子路径不等式: 保证 $\delta[w] \leq \delta[y]$ (否则 $x \rightarrow w \rightarrow y$ 可能比全局最短路径更长)
- 松弛有效性: 确保 $\lambda[w] \leq \lambda[x] + \text{length}(x, w)$ 不会被负权边破坏
- 反例: 若 $\text{length}(x, w) < 0$, 可能导致 $\delta[w] > \delta[y]$, 破坏归纳链条

朴素的数组实现的 dijkstra 算法的复杂度是 $O(n^2 + m)$, 如果用堆进行优化, 时间复杂度是 $O((n + m) \log n)$ (n 是顶点数量, m 是边数量)。如果是稠密图的话, 会使得堆优化速度更慢, 所以在稠密图 (邻接矩阵存储) 情况下, 应该用前者。

1.2.2 Kruskal 算法

对于 Kruskal 算法, 算法的时间复杂度取决于排序步骤, 所以是 $O(m \log m)$ 。

- $G = (V, E)$: 带权无向连通图
- T_{mst} : 任意一棵最小生成树
- A_k : 算法在第 k 步迭代后加入的边集 (初始化 $A_0 = \emptyset$)

$\forall k \geq 0, A_k \subseteq T_{\text{mst}}$ (即 A_k 总是某个 MST 边的子集)

1. 基础步骤 ($k = 0$):

$A_0 = \emptyset$, 空集是任何集合的子集, 显然 $A_0 \subseteq T_{\text{mst}}$ 。

2. 归纳步骤:

假设 $A_k \subseteq T_{\text{mst}}$, 设 $e = (u, v)$ 是第 $k + 1$ 步选择的边:

- 根据算法: e 是当前权重最小且不产生环的边 (即 e 连接 A_k 中两个不同连通分量)
- 根据安全边引理: $A_k \cup \{e\}$ 是某个 MST 边的子集

$\therefore A_{k+1} = A_k \cup \{e\} \subseteq T'_{\text{mst}}$ (T'_{mst} 为新的最小生成树)

3. 终止条件:

当 $|A_{\text{final}}| = |V| - 1$ 时:

- A_{final} 无环 (算法保证)
- A_{final} 连通所有顶点 (否则存在连接不同分量的安全边, 算法不会终止)
- $A_{\text{final}} \subseteq T_{\text{mst}}$ 且 $|A_{\text{final}}| = |T_{\text{mst}}|$

$\Rightarrow A_{\text{final}}$ 即为一棵最小生成树

下面用反证法证明安全边引理

假设存在权重最小的跨分量边 $e \notin T_{\text{mst}}$ (T_{mst} 是包含 A 的最小生成树)

1. 将 e 加入 T_{mst} , 形成唯一环 C (因树加边必成环)

2. 环 C 上必存在边 $e' = (x, y)$ 满足:

- $e' \neq e$
- e' 连接 A 的不同连通分量 (因 e 连接两个分量)
- $w(e') \geq w(e)$ (因 e 是权重最小的跨分量边)

3. 构造新生成树: $T_{\text{new}} = T_{\text{mst}} \cup \{e\} \setminus \{e'\}$

- 总权重: $w(T_{\text{new}}) = w(T_{\text{mst}}) + w(e) - w(e') \leq w(T_{\text{mst}})$
- 当 $w(e) < w(e')$ 时: $w(T_{\text{new}}) < w(T_{\text{mst}})$ (与 T_{mst} 最小性矛盾)
- 当 $w(e) = w(e')$ 时: T_{new} 也是最小生成树, 且包含 $A \cup \{e\}$

1.2.3 Prim 算法

Prim 算法一般复杂度为 $\Theta(m + n^2)$, 可以被堆优化为 $O(m \log n)$.

给出一个含权无向图 $G = (V, E)$, 其中 $|V| = n, |E| = m$:

- 算法 MST 可在 $O(m \log n)$ 时间内找出最小生成树
- 稠密图特例: 若存在 $\varepsilon > 0$ 使得 $m \geq n^{1+\varepsilon}$, 则时间复杂度可改进为 $O\left(\frac{m}{\varepsilon}\right)$

设 ε 为固定常数，其技术意义体现在：

$$\frac{m \log n}{m/\varepsilon} = \varepsilon \log n \Rightarrow \begin{cases} \text{当 } m = n^{1+\varepsilon} \text{ 时, 加速比为 } \Theta(\varepsilon \log n) \\ \varepsilon = \Omega(1) \text{ 时, } O\left(\frac{m}{\varepsilon}\right) \text{ 严格优于 } O(m \log n) \end{cases} \quad (1)$$

1. 稠密度控制： ε 量化图的稠密程度

$$m = n^{1+\varepsilon} \implies \text{边数 } m \text{ 远超顶点数 } n$$

2. 复杂度改进：当 ε 为常数时

$$O\left(\frac{m}{\varepsilon}\right) = O(m) \quad (\text{线性时间优于 } O(m \log n))$$

3. 实际意义：对满足 $\exists c > 0, m > n^{1+c}$ 的图，算法有显著加速

该定理的正确性建立在图论（稀疏/稠密图分类）与数据结构（斐波那契堆优化）的交叉分析上，其核心在于发现：当边数足够多时（ $m \gg n$ ），可以通过特殊数据结构设计消除对数因子。

Prim 算法可以正确处理包含负权边的图的最小生成树 (MST) 问题，其正确性不受边权值符号的影响。

证明. 设 $T_{\min}$ 是 G 的最小生成树。将 T 中的 $n-1$ 条边按权重从小到大排列为 $e_{k_1}, e_{k_2}, \dots, e_{k_{n-1}}$ ，将 $T_{\min}$ 中的边同样排列为 $e_{g_1}, e_{g_2}, \dots, e_{g_{n-1}}$ 。从头开始比较：

设第一条属于 T 但不属于 $T_{\min}$ 的边为 $e_{k_i} = (u_i, v_i)$ ，则在 $T_{\min}$ 中存在一条属于 $T_{\min}$ 但不属于 T 的边 $e_{g_j} = (u_i, v_*)$ 。根据 Prim 算法的性质 (2) 可知：

$$w(e_{k_i}) \leq w(e_{g_j})$$

在 $T_{\min}$ 中删除 e_{g_j} 并加入 e_{k_i} 得到新树 T_{new} ，满足：

$$\text{cost}(T_{\min}) \geq \text{cost}(T_{\text{new}})$$

由于 $T_{\min}$ 是最小生成树，又有：

$$\text{cost}(T_{\min}) \leq \text{cost}(T_{\text{new}})$$

故必然有：

$$\text{cost}(T_{\min}) = \text{cost}(T_{\text{new}})$$

即 T_{new} 也是最小生成树。

重复此替换过程直至 $T_{\text{new}} = T$ ，最终证明 T 是最小生成树。 $\square$

1.2.4 活动选择问题

你有 n 个活动构成的集合 $S = \{a_1, a_2, \dots, a_n\}$ ，这些活动都需要占用会议室，且每个时刻会议室只能给一场活动提供服务。每个活动 a_i 的开始时间 (start time) 为 s_i ，结束时间 (finish time) 为 f_i ，其中 $0 \leq s_i < f_i < \infty$ 。如果活动 a_i 占用的时间为左闭右开区间 $[s_i, f_i)$ 。如果活动 a_i 和 a_j 的时间区间 $[s_i, f_i)$ 和 $[s_j, f_j)$ 没有重叠，那么活动 a_i 和 a_j 相互兼容。活动选择问题目标是从所有活动中选出一个总时间规模最大相互兼容的活动子集。假设这些活动已经按照结束时间从小到大排序，即： $f_1 \leq f_2 \leq f_3 \leq \dots \leq f_n$

如果给定活动序列没有按照结束时间从小到大排序，先进行排序，这步预处理非常重要，没有这步预处理，后面的贪心策略就无从开展。(其实也可以选择最晚开始的活动，思路与最早结束大体一致，不再额外展开)

下面开始使用贪心策略求解，贪心策略每一步都要作出贪心选择 (greedy choice)，这样只有一个子问题需要处理。下面会分别讨论贪心的递归算法和迭代算法。

下面证明活动选择问题具有最优子结构。

定义 S_{ij} 为 a_i 结束后 a_j 开始前的活动集合，定义 A_{ij} 为 S_{ij} 中规模最大且相互兼容的活动子集。 A_{ij} 中包含活动 a_k ，和两个子问题 A_{ik} 和 A_{kj} 。定义 $A_{ik} = A_{ij} \cap S_{ik}$ 且 $A_{kj} = A_{ij} \cap S_{kj}$ ，这样有 $A_{ij} = A_{ik} \cup \{a_k\} \cup A_{kj}$ ，所以有 $|A_{ij}| = |A_{ik}| + |A_{kj}| + 1$ 。

用剪切粘贴方法可以证明最优解 A_{ij} 一定包含两个子问题 S_{ik} 和 S_{kj} 的最优解。如果你能找到一个由 S_{kj} 相互兼容的活动组成的集合 A'_{kj} ，其中 $|A'_{kj}| > |A_{kj}|$ ，那么你就可以用 A'_{kj} 替换 A_{kj} 。此时就能构造出规模为

$$|A_{ik}| + |A'_{kj}| + 1 > |A_{ik}| + |A_{kj}| + 1 = |A_{ij}|$$

的相互兼容的活动集合，与 A_{ij} 是最优解矛盾。同理也可以用于证明子问题 S_{ik} 。

定义 S_{ij} 的最优解的规模为 $c[i, j]$ ，运用动态规划方法可以写出递归式： $c[i, j] = c[i, k] + c[k, j] + 1$ ，即：

$$c[i, j] = \begin{cases} 0 & S_{ij} = \emptyset \\ \max_{a_k \in S_{ij}} \{c[i, k] + c[k, j] + 1\} & S_{ij} \neq \emptyset \end{cases}$$

虽然我们可以用动态规划方法求解，但是我们忽略了活动选择问题的一个重要特征，这个特征可以大大提高问题求解速度。

考虑任意非空子问题 S_k ，如果 a_m 是 S_k 中结束时间最早的活动，那么 a_m 在 S_k 的某个最大兼容活动子集中。

下面证明活动选择问题具有贪心选择性质：证明：定义 A_k 是 S_k 的最大兼容活动子集，且 a_j 是 A_k 中结束时间最早的活动。

若 $a_j = a_m$ ，则已证明 $a_m \in A_k$ 。

若 $a_j \neq a_m$ ，令 $A'_k = (A_k - \{a_j\}) \cup \{a_m\}$ ，则 $|A_k| = |A'_k|$ 。又 a_j 是 A_k 中结束时间最早的活动， a_m 是 A'_k 中结束时间最早的活动， $f_m \leq f_j$ 。所以 A'_k 是 S_k 的最大兼容活动子集， $a_m \in A'_k$ 。

我们每次选择结束时间最早的且与当前活动兼容的活动，重复执行这个过程直到没有剩余活动可以选择。所选择的活动的结束时间必然是严格递增的。所以我们只需要按照结束时间单调递增的顺序处理所有活动，每个活动仅需要考察一次。

Input: $s[1..n]$: 活动开始时间数组 $f[1..n]$: 活动结束时间数组 (已按 f_i 升序排列) k : 当前活动索引 (初始调用时 $k = 0$) n : 总活动数**Output:** 最大兼容活动子集

```

1 function RECURSIVE-ACTIVITY-SELECTOR( $s, f, k, n$ )
2    $m \leftarrow k + 1$ 
3   while  $m \leq n$  and  $s[m] < f[k]$  do
4      $m \leftarrow m + 1$  // 在  $S_k$  中寻找第一个兼容活动
5   end
6   if  $m \leq n$  then
7     return  $\{a_m\} \cup \text{RECURSIVE-ACTIVITY-SELECTOR}(s, f, m, n)$ 
8   end
9   else
10    return  $\emptyset$ 
11  end
12 // 初始调用方式
13 RECURSIVE-ACTIVITY-SELECTOR( $s, f, 0, n$ )

```

Algorithm 3: 递归式活动选择算法

当然也可以用动态规划的思想实现, 同时用二分的思路对其进行优化, 时间复杂度仍为 $O(n \log n)$:

思考活动选择问题的一个变型: 每个活动 a_i 除了开始时间和结束时间外, 还有一个权值 v_i 。目标不再是求规模最大的活动子集, 而是求权值总和最大的活动子集, 也就是选择一个活动集合 A , 使得 $\sum_{a_k \in A} v_k$ 最大化。设计一个多项式时间的算法求解该问题。

其实就只需要修改一下动态规划算法的比较项就可以了, 不再做更多展开。

1.2.5 区间图着色问题

假设有一组活动, 将它们安排到一些讲堂, 任意活动都可以在任意讲堂进行, 我们希望用最少的教室完成所有活动, 设计一个贪心算法求每个活动所在讲堂进行分配。

这个问题可以视为区间图着色问题 (interval-graph coloring problem)。区间图着色问题: 构造一个区间图, 顶点表示给定的活动, 边连接不兼容的活动。要求用最少的颜色对顶点进行着色, 相邻顶点的颜色不能相同。

一个朴素的方法是, 每当有新活动时, 就遍历所有讲堂, 查看是否有空闲讲堂。如果没有空闲讲堂, 就新开一个讲堂。为了方便选出空闲讲堂, 我们可以使用优先队列。

算法执行步骤如下:

按照开始时间对活动进行排序。初始化一个小顶堆, 按照结束时间排序, 堆顶活动为最早结束的活动。检查每一个活动: 如果堆为空, 开辟一个新讲堂, 选好讲堂后将活动加入堆中, 同时将该活动记录到所在讲堂举行的活动名单中。如果堆不为空, 和堆顶活动进行比较, 如果堆顶活动的结束时间小于或等于当前活动的开始时间, 将堆顶活动出队, 并将当前活动放在堆顶活动所在讲堂进行, 否则开辟一个新讲堂。选好讲堂后将活动加入堆中, 同时将该活动记录到所在讲堂举行的活动名单中。所有活动检查结

Input: $s[1..n]$: 活动开始时间数组 $f[1..n]$: 活动结束时间数组（已按 f_i 升序排列） n : 总活动数**Output:** 最大兼容活动集大小

```

1 function DP-ACTIVITY-SELECTOR( $s, f, n$ )
2   let  $c[0..n]$  be new array
3    $c[0] \leftarrow 0$ 
4   for  $i \leftarrow 1$  to  $n$  do
5      $l \leftarrow 1, r \leftarrow n - 1$ 
6     while  $l \leq r$  do
7        $mid \leftarrow \lfloor (l + r) / 2 \rfloor$ 
8        $f[mid] \leq s[i]$ 
9        $\Rightarrow$ 
10       $l \leftarrow mid + 1$ 
11       $f[mid] > s[i]$ 
12       $\Rightarrow$ 
13       $r \leftarrow mid - 1$ 
14    end
15     $k \leftarrow r$ 
16     $c[i] \leftarrow \max(c[k] + 1, c[i - 1])$ 
17 end
18 return  $c[n]$ 

```

Algorithm 4: 动态规划活动选择算法

束后，输出所有讲堂举行的活动名单。

1.2.6 最小延迟调度问题

给定 n 个作业，每个作业 J_i 有一个处理时间 t_i 和截止时间 d_i 。调度方案需为作业排列一个顺序，使得所有作业的延迟（完成时间减去截止时间）的最大值最小化。形式化定义为：

- 作业顺序 $\sigma = (J_{\sigma(1)}, \dots, J_{\sigma(n)})$ 的完成时间 $C_{\sigma(i)} = \sum_{k=1}^i t_{\sigma(k)}$ 。
- 延迟 $L_{\sigma(i)} = \max(0, C_{\sigma(i)} - d_{\sigma(i)})$ 。
- 目标是 minimize $L_{\max} = \max_i L_{\sigma(i)}$ 。

Input: 一组活动 $A = \{a_1, a_2, \dots, a_n\}$, 其中每个活动 a_i 有开始时间 s_i 和结束时间 f_i

Output: 分配讲堂的列表, 每个讲堂包含一组不重叠的活动

```
1 按照开始时间  $s_i$  对活动进行升序排序;
2 初始化一个最小堆  $H$ , 按结束时间排序;
3 初始化空的讲堂列表  $classrooms$ ;
4 foreach 活动  $a_i \in A$  do
5   if  $H$  不为空 且  $H$  的堆顶活动  $a_{top}$  满足  $f_{top} \leq s_i$  then
6     从堆  $H$  中弹出  $a_{top}$ ;
7     将  $a_i$  分配到  $a_{top}$  所在的讲堂  $c$ ;
8   end
9   else
10    创建新讲堂  $c_{new}$ ;
11    将  $a_i$  分配到  $c_{new}$ ;
12    将  $c_{new}$  加入  $classrooms$ ;
13  end
14 将  $a_i$  的结束时间  $f_i$  插入堆  $H$ ;
15 end
16 return  $classrooms$ ;
```

Algorithm 5: 最小讲堂分配算法 (区间图着色)

Input: 任务集合 $S = \{J_1, J_2, \dots, J_n\}$, 其中每个任务 J_i 有处理时间 t_i 和截止时间 d_i

Output: 任务完成时间数组 f

```
1 按照截止时间  $d_i$  对任务进行升序排序, 使得  $d_1 \leq d_2 \leq \dots \leq d_n$ ;
2 初始化完成时间数组:  $f(1) \leftarrow 0$ ;
3 初始化计数器:  $i \leftarrow 2$ ;
4 while  $i \leq n$  do
5    $f(i) \leftarrow f(i-1) + t_{i-1}$ ;
6    $i \leftarrow i + 1$ ;
7 end
8 return  $f$ ;
```

Algorithm 6: 最小延迟调度算法

显然, 所有无逆序且无空闲时间的调度具有相同的最大延迟。即对于满足 $d_{\sigma(1)} \leq \dots \leq d_{\sigma(n)}$ 的调度 σ , 其 $L_{\max}$ 仅取决于任务顺序, 与相同截止时间任务的内部排列无关。

下面证明: 按 d_i 升序排列 (EDD 策略) 的调度 σ^* 是最优解。

若 $d_i > d_j$ 但 i 排在 j 前, 则称相邻任务对 (J_i, J_j) 为逆序对。

对任意含逆序对 (J_i, J_j) 的调度 σ :

$$\text{交换前延迟: } L_i = C + t_i - d_i, \quad L_j = C + t_i + t_j - d_j$$

$$\text{交换后延迟: } L'_j = C + t_j - d_j, \quad L'_i = C + t_j + t_i - d_i$$

其中 C 为 J_i 的开始时间。由于 $d_i > d_j$ 且 $t_j > 0$:

$$\max(L'_j, L'_i) \leq \max(L_i, L_j)$$

通过有限次相邻交换可消除所有逆序对:

- 每次交换不增加 $L_{\max}$
- 最终得到无逆序的 EDD 调度
- 由引理 1 知此调度最优

1.2.7 分数背包问题

给出 n 个大小为 $s_1, s_2, \dots, s_n$, 值为 $v_1, v_2, \dots, v_n$ 的物品, 并设背包的容量为 C 。要求找到非负实数 $x_1, x_2, \dots, x_n$, 使得在满足不超过背包总容量的约束条件下, 装进背包物品部分总价值最大。

即, 最大化 $\sum_{i=1}^n x_i v_i$ 且满足 $\sum_{i=1}^n x_i s_i \leq C$

选择单位大小的价值最大的物品即可, 不过多展开了 (过于简单)

1.2.8 带期限作业排序问题

问题: 设有一个单机系统、无其它资源限制且每个作业运行相等时间, 不妨假定每个作业运行 1 个单位时间。现有 n 个作业, 每个作业都有一个截止期限 $d_i > 0$, d_i 为整数。如果作业能够在截止期限之内完成, 可获得 $p_i > 0$ 的收益。问题要求得到一种作业调度方案, 该方案给出作业的一个子集和该作业子集的一种排列, 使得若按照这种排列次序调度作业运行, 该子集中的每个作业都能如期完成, 并且能够获得最大收益。

思路: 将输入的作业按收益从大到小进行排序, 用一个数组 vis 表示某时刻是否已经有作业正在运行, $vis[i] = 1$, 表示时间 i 被占用。然后从第 1 个作业开始往后搜索, 将该作业安排到 $vis[d]$ 时间运行, 如果 d 已经有安排, 将从 $d-1$ 开始往前搜索, 直到找到一个未被占用的时间点。如果找不到空时间点, 则跳过此作业, 继续向后搜索直到所有作业都搜索完。

Input: 作业集合 $J = \{J_1, \dots, J_n\}$, 其中每个作业 J_i 包含:

- 编号 id_i
- 收益 $p_i > 0$
- 截止期限 $d_i > 0$ (整数)

Output: 能在截止期限前完成的作业子集及其排列顺序

```
1 按照收益  $p_i$  对作业进行降序排序;
2 初始化时间槽数组  $vis[1..max\_d]$  全为 0;
3 初始化结果列表  $res \leftarrow \emptyset$ ;
4 foreach 作业  $J_i \in J$  (按已排序顺序) do
5    $found \leftarrow false$ ;
6   if  $vis[J_i.d] == 0$  then
7      $vis[J_i.d] \leftarrow 1$ ;
8      $res.push\_back(J_i.id)$ ;
9      $found \leftarrow true$ ;
10  end
11  else
12    for  $t \leftarrow J_i.d - 1$  downto 1 do
13      if  $vis[t] == 0$  then
14         $vis[t] \leftarrow 1$ ;
15         $res.push\_back(J_i.id)$ ;
16         $found \leftarrow true$ ;
17        break;
18      end
19    end
20  end
21  if not found then
22    跳过作业  $J_i$ ;
23  end
24 end
25 return  $res$ ;
```

Algorithm 7: 最大收益作业调度算法

定理 1. 按效益值降序排列的贪心调度算法总能得到最优解。

证明. 采用交换论证法, 分三部分证明:

1. 基本设定:

- 设最优解作业集为 I , 贪心解为 J
- 若 $I = J$ 则得证, 故只需考虑 $I \neq J$ 的情况
- 此时必有 $I \not\subseteq J$ (否则 I 非最优) 且 $J \not\subseteq I$ (否则 J 更优)

- 存在 $a \in J - I$ 和 $b \in I - J$ ，且由贪心法性质有 $p_a \geq p_b$

2. 调度表调整:

1. 设 S_J 和 S_I 分别为 J 和 I 的可行调度表
2. 对公共作业 $i \in I \cap J$ ，调整调度表使得：
 - 若在 S_J 中 i 于 $[t, t+1]$ 执行，在 S_I 中于 $[t', t'+1]$ 执行
 - 当 $t < t'$ 时：将 S_J 中 $[t', t'+1]$ 的作业与 i 交换
 - 当 $t' < t$ 时：对 S_I 做对称调整
3. 最终得到新调度表 S'_J 和 S'_I ，使公共作业执行时间一致

3. 最优性保持:

- 在调整后的 S'_I 中，将作业 b 替换为 a ，形成新解 $I' = (I - \{b\}) \cup \{a\}$
- 由于 $p_a \geq p_b$ ，有 $\sum_{i \in I'} p_i \geq \sum_{i \in I} p_i$
- 重复此过程可将 I 逐步转换为 J 而不降低总效益

因此 J 必为最优解。 □

定理 2. 作业集合 J 可行的充要条件：存在按 d_i 非降序排列的调度序列 σ ，使得 $\forall i \in J$ 都在 d_i 前完成。

证明. 必要性由定义显然。充分性证明：

- 设 σ 为按 d_i 非降序排列的调度
- 若存在作业 i_k 在 σ 中第 k 个执行且完成时间 $k > d_{i_k}$
- 则前 k 个作业的 $d_{i_j} \leq d_{i_k} < k$ ，导致至少一个作业必然超期
- 矛盾，故 σ 必为可行调度 □

1.2.9 黑白点最大匹配问题

Input: 黑点集合 $B = \{b_1, \dots, b_n\}$, 其中 $b_i = (x_i^B, y_i^B)$
 白点集合 $W = \{w_1, \dots, w_n\}$, 其中 $w_j = (x_j^W, y_j^W)$
Output: 最大匹配数 $count$

```

1 预处理阶段: 将黑点  $B$  按  $x^B$  升序排列 ( $x$  相同则按  $y^B$  升序);
2 将白点  $W$  按  $x^W$  降序排列 ( $x$  相同则按  $y^W$  降序);
3 匹配阶段:  $count \leftarrow 0$ ;
4  $i \leftarrow 1, j \leftarrow 1$ ;
5 while  $i \leq n$  且  $j \leq n$  do
6   if  $x_i^B \geq x_j^W$  且  $y_i^B \geq y_j^W$  then
7      $count \leftarrow count + 1$ ;
8      $i \leftarrow i + 1$ ; // 消耗当前黑点
9      $j \leftarrow j + 1$ ; // 消耗当前白点
10  end
11  else
12     $j \leftarrow j + 1$ ; // 跳过无法匹配的白点
13  end
14 end
15 return  $count$ ;

```

Algorithm 8: 黑白点最大匹配算法

证明. 我们通过以下步骤证明:

- 设最优匹配为 M^* , 其匹配数为 OPT
- 设算法得到的匹配为 M , 匹配数为 $|M|$
- 定义特征函数 $\phi(M)$ 为匹配中黑点的字典序排列

对于任意最优匹配 M^* , 存在匹配 M' 使得:

1. $|M'| = |M^*|$
2. $\phi(M')$ 与算法输出 M 的字典序相同 (可以通过调整使得相同)

假设存在更优匹配 M^* , 取第一个差异点:

1. 设算法第 k 步选择 (b_i, w_j)
2. 而 M^* 选择 $(b_i, w_{j'})$ 或 $(b_{i'}, w_j)$ (如果不是则通过交换进行修正)
3. 情况 1: $w_{j'} \neq w_j$

由于算法优先选最大的 w_j

$$\begin{aligned} \Rightarrow x_j^W &\geq x_{j'}^W \\ y_j^W &\geq y_{j'}^W \end{aligned}$$

可交换 w_j 和 $w_{j'}$ 而不减少匹配数

4. 情况 2: $b_{i'} \neq b_i$

算法选择最小的 b_i

$$\begin{aligned} \Rightarrow x_i^B &\leq x_{i'}^B \\ y_i^B &\leq y_{i'}^B \end{aligned}$$

用 b_i 替换 $b_{i'}$ 可保持可行性

上述交换均会使 M^* 的特征向量 $\phi(M^*)$ 更接近算法结果 M , 与 M^* 的最优性假设矛盾。 $\square$

1.2.10 过河问题

Oliver 和 $N-1$ 个朋友共 N 人需要从河东岸渡到西岸。他们只有一艘小船, 每次最多可载两人。当两个人 i 和 j 一起渡河时, 所需时间为 $\max(T_i, T_j)$ 。求所有人过河的最少总时间。

很明显的贪心, 主要讨论两种情况: 1、让速度最快的往返接送, 每次送两个人所用时间为两倍最快的人所用的时间加上两个最慢的人的所用的时间之和。即 $a[n] + a[1] + a[n-1] + a[1]$;

2、先让两个最快的人过河, 再让第二快 (谁将船送回都不影响) 的人送回船, 让两个岁慢的人过河, 再将留在河对面的速度最快的人把船送回。所用时间为 $a[2] + a[2] + a[n] + a[1]$;

1.2.11 树上的最小顶点覆盖

给出一种有效的贪心算法, 该算法能在线性时间内找到一棵树中的一个最优顶点覆盖。

表示方案一采用定长数组存储每个结点的孩子结点, 设给定一棵树 T , 其根结点为 $T.root$, 阶数为 $T.degree$ 。对于每个结点 x , 具有颜色属性 $x.color$ 和包含 $T.degree$ 个孩子结点指针的数组 $x.children$ 。该方案借鉴红黑树思想进行结点着色, 规定每个结点非红即黑, 所有空结点默认为黑色, 且红色结点的所有孩子结点必须为黑色。在贪心算法实现中, 首先将所有孩子结点为空结点的非空结点标记为红色, 然后采用自底向上的遍历策略。具体实现采用方法一的后序遍历方式, 其着色规则为: 所有空结点保持黑色; 对于非空结点, 若其所有孩子结点均为黑色则将该结点着为红色, 否则着为黑色。通过自底向上的后序遍历过程, 当所有结点完成着色时, 最终所有黑色结点的集合 C 即构成该树的最优顶点覆盖解。该方法的正确性依赖于树的后序遍历特性与颜色约束规则的结合, 确保在多项式时间内获得满足条件的顶点覆盖。

Input: n : number of people, $t[1..n]$: array of crossing times

Output: Minimum total time required

```
1 Sort  $t[1..n]$  in ascending order;
2 Initialize  $ans \leftarrow 0$ ,  $lef \leftarrow n$ ;
3 while  $lef > 0$  do
4   if  $lef == 1$  then
5      $ans \leftarrow ans + t[1]$ ;
6     Break;
7   else
8     if  $lef == 2$  then
9        $ans \leftarrow ans + t[2]$ ;
10      Break;
11    else
12      if  $lef == 3$  then
13         $ans \leftarrow ans + t[1] + t[2] + t[3]$ ;
14        Break;
15      else
16         $s1 \leftarrow t[1] \times 2 + t[lef] + t[lef - 1]$ ;
17         $s2 \leftarrow t[1] + t[2] \times 2 + t[lef]$ ;
18         $ans \leftarrow ans + \min(s1, s2)$ ;
19         $lef \leftarrow lef - 2$ ;
20      end
21    end
22  end
23 end
24 return  $ans$ ;
```

Algorithm 9: Minimum Time River Crossing